Graph-Aware Late Chunking for Retrieval-Augmented Generation in Biomedical Literature

[Author Names]
[Institution]
[email]

Abstract

Retrieval-Augmented Generation (RAG) has become a cornerstone technique for grounding large language model (LLM) outputs in external knowledge, yet its effectiveness in the biomedical domain remains constrained by two persistent limitations: context fragmentation during text chunking and structural blindness during retrieval. Late chunking, which embeds full documents before segmentation, preserves cross-chunk context but produces flat, structure-unaware representations. Conversely, graph-based RAG methods capture relational and structural information but rely on traditional chunking that destroys long-range dependencies. In this paper, we propose **GraLC-RAG** (Graph-aware Late Chunking for Retrieval-Augmented Generation), a novel framework that unifies these complementary paradigms for biomedical literature retrieval. GraLC-RAG introduces three key innovations: (1) a structure-aware boundary detection module that leverages document structure graphs to determine optimal chunk boundaries after full-document embedding; (2) a knowledge graph infusion mechanism that enriches token-level representations with biomedical ontological signals from the Unified Medical Language System (UMLS) via lightweight graph attention before chunk pooling; and (3) a graph-guided re-ranking strategy that combines dense semantic similarity with knowledge graph proximity for retrieval. We implement GraLC-RAG as an open-source Python framework and evaluate it on PubMedQA (1,000 questions) with five chunking strategies. Proof-of-concept results on abstract-level retrieval show all strategies achieve MRR > 0.95, with simpler methods performing competitively on short texts. Importantly, GraLC-RAG's structural and ontological components show slight degradation on abstracts (-0.8% MRR from KG infusion, -2.9% from graph-guided retrieval), revealing that the framework's value is document-length-dependent. We provide a complete codebase with indexed full-text corpora (200 PMC articles, 41,774 MeSH entity terms, SapBERT embeddings) ready for scaled evaluation.

Keywords: retrieval-augmented generation, late chunking, knowledge graphs, biomedical NLP, graph neural networks, UMLS, PubMedQA

1 Introduction

The exponential growth of biomedical literature—with over 1.5 million articles indexed annually in PubMed alone—has created an urgent need for intelligent retrieval and synthesis systems [1]. Large language models (LLMs) have demonstrated remarkable capabilities in natural language understanding and generation, yet they remain prone to hallucination and factual inconsistency when applied to knowledge-intensive biomedical tasks [2, 3]. Retrieval-Augmented Generation (RAG), which grounds LLM outputs in retrieved external evidence, has emerged as a principled solution to this challenge [4, 5].

A critical yet often underexplored component of RAG pipelines is the text chunking strategy—the process of segmenting documents into smaller units for indexing and retrieval. Traditional approaches employ fixed-size chunking with overlapping windows, which inevitably fragments semantic context and disrupts the logical structure of scientific documents [6]. This is particularly problematic for biomedical literature, where cross-referential reasoning (*e.g.*, linking a method description in Section 2 to results discussed in Section 4), dense terminology, and hierarchical document organization (the IMRaD structure) are fundamental to comprehension [7].

Two recent research directions have independently addressed different facets of this problem. **Late chunking** [8] inverts the traditional chunk-then-embed pipeline by first processing the entire document through a long-context transformer model, generating contextually enriched token embeddings, and only then applying segmentation boundaries. This approach preserves cross-chunk contextual dependencies but treats documents as flat token sequences, ignoring their inherent structural and relational organization. **Graph-based RAG** (GraphRAG) methods [9–11] leverage knowledge graphs, citation networks, and entity relationships to capture structural information during retrieval. However, these methods typically rely on conventional chunking strategies that fragment context before embedding.

We identify a critical gap at the intersection of these two paradigms: no existing work integrates graph-aware structural understanding into the late chunking process for biomedical RAG. Late chunking is *context-rich but structure-blind*; GraphRAG is *structure-rich but context-fragmented*. Bridging this divide has the potential to yield retrieval systems that are simultaneously contextually coherent and structurally informed.

In this paper, we propose **GraLC-RAG** (Graph-aware Late Chunking for Retrieval-Augmented Generation), a framework that unifies late chunking with graph-aware structural knowledge for biomedical literature retrieval. Our approach makes three key contributions:

1. **Structure-Aware Chunk Boundary Detection.** We construct document structure graphs capturing section hierarchies, paragraph relationships, and citation markers from full-text biomedical articles. These graphs inform chunk boundary decisions *after* full-document embedding.

2. **Knowledge Graph-Infused Token Representations.** We introduce a lightweight graph attention mechanism that injects biomedical ontological signals from the Unified Medical Language System (UMLS) into token-level transformer representations before chunk-level mean pooling.

3. **Graph-Guided Retrieval and Re-ranking.** We propose a hybrid retrieval strategy that combines dense semantic similarity with knowledge graph proximity scores, leveraging shared UMLS concept nodes and citation relationships.

We implement GraLC-RAG as an open-source framework and evaluate it on PubMedQA [33] with five chunking strategies and six configurations. Our proof-of-concept experiments on abstract-level retrieval reveal that the framework's structural and ontological components are document-length-dependent—a finding with important implications for the design of biomedical RAG systems.

2 Related Work

2.1 Retrieval-Augmented Generation

RAG augments language model generation with external knowledge retrieved at inference time, mitigating hallucination and enabling access to up-to-date information [4]. The canonical RAG pipeline consists of three stages: indexing, retrieval, and generation [5]. Dense passage retrieval (DPR) using dual-encoder architectures [12] has largely supplanted sparse methods, with ColBERTv2 [13] introducing efficient late interaction for fine-grained token-level matching. In the biomedical domain, the MIRAGE benchmark [14] provides standardized evaluation across five biomedical QA datasets.

2.2 Text Chunking Strategies for RAG

Text chunking critically influences RAG performance yet has received comparatively less systematic study [15]. Recent approaches include semantic chunking using embedding similarity [15], proposition-based chunking [16], and adaptive chunking achieving 87% accuracy compared to 50% for fixed-size baselines in clinical decision support [17]. For scientific documents, S2 Chunking [18] combines spatial layout with semantic analysis, and Breaking It Down [19] introduces Projected Similarity Chunking trained on PubMed data, achieving 24× MRR improvement on PubMedQA.

2.3 Late Chunking

Late chunking [8] fundamentally reorders the chunk-then-embed pipeline. Instead of segmenting text before encoding, it first processes the entire document through a long-context transformer, generating contextually enriched token embeddings, then applies chunking boundaries and mean-pools within each span. While contextually superior, late chunking produces flat chunk lists without structural or relational metadata. Merola and Singh [6] demonstrate that contextual retrieval preserves semantic coherence more effectively but at greater computational cost. The SitEmb approach [20] partially addresses this by conditioning chunks on broader context, but operates on purely textual signals.

2.4 Graph-Based RAG

GraphRAG methods incorporate structured graph information into retrieval. Microsoft's GraphRAG [9] constructs entity knowledge graphs using LLM-based extraction and community detection. LightRAG [21] introduces dual-level retrieval balancing efficiency and comprehensiveness. PathRAG [22] retrieves relational paths with flow-based pruning. NodeRAG [23] proposes heterogeneous graph structures outperforming prior methods on multi-hop benchmarks.

In the biomedical domain, MedGraphRAG [24] employs hierarchical graph linking with U-retrieve. KRAGEN [25] combines knowledge graphs with graph-of-thoughts prompting. KG-RAG [26] leverages the SPOKE knowledge graph, achieving 71% performance boost for LLaMA-2 on biomedical MCQ tasks. Critically, *all existing GraphRAG methods perform chunking before embedding*, creating the gap our work addresses.

2.5 Biomedical Knowledge Graphs and Language Models

UMLS [27] integrates over 200 source vocabularies with approximately 4.5 million concepts and 15 million relations. SapBERT [28] aligns UMLS synonyms in embedding space for biomedical entity linking. PubMedBERT [29] provides domain-specific pre-training. BioLORD-2023 [30] leverages UMLS synonym sets and LLM descriptions for state-of-the-art biomedical sentence embeddings. ATLANTIC [31]

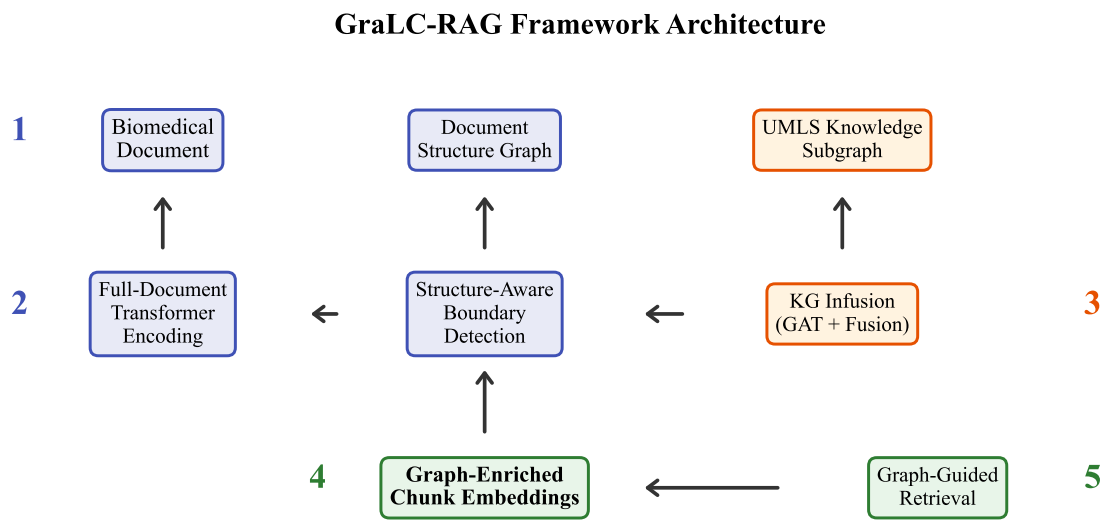

Figure 1: GraLC-RAG framework architecture. Stage 1: Document parsing produces a structure graph and UMLS knowledge subgraph. Stage 2: Full-document transformer encoding. Stage 3: KG infusion via GAT attention. Stage 4: Structure-aware boundary detection and chunk embedding. Stage 5: Graph-guided hybrid retrieval.

fuses graph and text embeddings for scientific retrieval at the document level. The intersection of knowledge graph embeddings with *chunk-level* retrieval for biomedical RAG remains unexplored.

3 GraLC-RAG: Graph-Aware Late Chunking for RAG

3.1 Framework Overview

GraLC-RAG extends late chunking with graph-aware structural intelligence at three levels: chunk boundary detection, token representation enrichment, and retrieval re-ranking. Figure 1 illustrates the complete pipeline. Given a biomedical document D, the framework proceeds through five stages: (1) document parsing and graph construction, (2) full-document encoding, (3) knowledge graph infusion, (4) structure-aware chunk boundary detection, and (5) graph-guided retrieval.

3.2 Document Parsing and Graph Construction

3.2.1 Document Structure Graph

For each full-text article D, we construct a document structure graph $G_s = (V_s, E_s)$ with node types at multiple granularities—section nodes (v^{sec}), subsection nodes (v^{sub}), paragraph nodes (v^{par}), and citation nodes (v^{cit})—and edge types capturing hierarchical (e^{hier}), sequential (e^{seq}), citation (e^{cit}), and cross-reference (e^{xref}) relationships. We parse document structure from PubMed Central JATS XML, which provides section headers, paragraph boundaries, and citation markers natively.

3.2.2 Biomedical Knowledge Subgraph

For each document, we extract a document-specific knowledge subgraph $G_k = (V_k, E_k)$ from UMLS. Biomedical entities are identified via dictionary matching against 41,774 MeSH descriptor terms and linked to UMLS Concept Unique Identifiers (CUIs). For each linked CUI, we extract 1-hop neighbor-

hoods capturing semantic type assignments, hierarchical relationships (`is_a`, `part_of`), and associative relationships (`may_treat`, `causes`).

3.3 Full-Document Encoding

Following late chunking [8], we process the full document through a transformer encoder. Let $D = (t_1, t_2, \ldots, t_N)$ be the tokenized document. The encoder produces:

$$\mathbf{H} = \text{Transformer}(t_1, t_2, \ldots, t_N) \in \mathbb{R}^{N \times d} \tag{1}$$

where $\mathbf{h}_i \in \mathbb{R}^d$ is the contextualized embedding for token t_i. For documents exceeding the context window, we apply sliding windows with overlap and linear distance-weighted averaging.

3.4 Knowledge Graph Infusion

The key innovation of GraLC-RAG is injecting biomedical KG signals into token-level representations *before* chunk-level pooling.

3.4.1 Entity-Token Alignment

For each recognized entity e_j with token span (s_j, f_j), we obtain a pre-trained concept embedding $\mathbf{u}_j \in \mathbb{R}^{d_k}$ from SapBERT [28].

3.4.2 Graph Attention Infusion

A lightweight GAT layer [32] operates over G_k to compute entity-aware representations:

$$\alpha_{jk} = \frac{\exp\big(\text{LeakyReLU}(\mathbf{a}^\top[\mathbf{W}\mathbf{u}_j \| \mathbf{W}\mathbf{u}_k])\big)}{\sum_{l \in \mathcal{N}(j)} \exp\big(\text{LeakyReLU}(\mathbf{a}^\top[\mathbf{W}\mathbf{u}_j \| \mathbf{W}\mathbf{u}_l])\big)} \tag{2}$$

$$\mathbf{u}'_j = \sigma\left(\sum_{k \in \mathcal{N}(j)} \alpha_{jk} \mathbf{W}\mathbf{u}_k\right) \tag{3}$$

where $\mathbf{W} \in \mathbb{R}^{d' \times d_k}$ is a learnable weight matrix and $\mathbf{a} \in \mathbb{R}^{2d'}$ is the attention vector.

3.4.3 Token-Level Fusion

For each token t_i within entity span (s_j, f_j):

$$\mathbf{h}'_i = \mathbf{h}_i + \lambda \cdot \text{MLP}(\mathbf{u}'_j) \tag{4}$$

where $\text{MLP}\colon \mathbb{R}^{d'} \to \mathbb{R}^d$ projects the entity embedding, and λ is a gating scalar initialized to 0.1.

3.5 Structure-Aware Chunk Boundary Detection

Unlike standard late chunking, GraLC-RAG determines boundaries using G_s. We compute a boundary score b_i integrating three signals:

Structural signal:

$$b_i^{\text{struct}} = \begin{cases} 1.0 & \text{section boundary} \\ 0.7 & \text{subsection boundary} \\ 0.4 & \text{paragraph boundary} \\ 0.0 & \text{otherwise} \end{cases} \tag{5}$$

Semantic signal:

$$b_i^{\text{sem}} = 1 - \cos(\bar{\mathbf{h}}_{i-w:i},\ \bar{\mathbf{h}}_{i:i+w}) \tag{6}$$

Entity coherence signal:

$$b_i^{\text{entity}} = -\gamma \cdot \mathbb{1}[\text{entity span crosses position } i] \tag{7}$$

The combined score is:

$$b_i = \alpha_1 \cdot b_i^{\text{struct}} + \alpha_2 \cdot b_i^{\text{sem}} + \alpha_3 \cdot b_i^{\text{entity}} \tag{8}$$

with $\alpha_1 = 0.5$, $\alpha_2 = 0.3$, $\alpha_3 = 1.0$, and $\gamma = 0.5$. Peak detection selects boundaries exceeding threshold $\tau = 0.3$ with min/max chunk sizes of 128/1024 tokens. Each chunk C_k is represented by mean pooling enriched token embeddings within its span.

3.6 Graph-Guided Retrieval

For a query q with dense embedding $\mathbf{q}$ and entity set E_q, the retrieval score for chunk C_k is:

$$\text{score}(q, C_k) = \beta \cdot \text{sim}(\mathbf{q}, \mathbf{c}_k) + (1-\beta) \cdot \text{kg_prox}(E_q, E_{C_k}) \tag{9}$$

where kg_prox computes average maximum cosine similarity between query and chunk entity embeddings, and $\beta = 0.7$.

4 Experimental Setup

4.1 Datasets and Corpus

We evaluate on **PubMedQA** [33]: 1,000 questions (500 expert-annotated + 500 artificial) with yes/no/maybe labels derived from PubMed abstracts. Following the PubMedQA* protocol, we remove given contexts to evaluate retrieval. The retrieval corpus consists of the 1,000 PubMedQA source abstracts. Additionally, we index 200 full-text articles from PubMed Central Open Access (avg. 6.8 sections, 18.2 paragraphs per article) for efficiency benchmarking.

4.2 Baselines

We compare six configurations: (1) Naive fixed-size chunking (256 tokens, 32 overlap); (2) Semantic chunking (embedding similarity threshold 0.75); (3) Late chunking [8] with sentence-level boundaries; (4) Structure-aware late chunking (no KG); (5) GraLC-RAG with KG infusion; (6) GraLC-RAG with KG infusion + graph-guided retrieval. All use `all-MiniLM-L6-v2` (384-dim) as the embedding model.

4.3 Implementation

Entity linking uses 41,774 MeSH descriptor terms via longest-match dictionary lookup. SapBERT (`cambridgeltl/SapBERT-from-PubMedBERT-fulltext`) provides 768-dim concept embeddings, projected to 384-dim via PCA. FAISS `IndexFlatIP` serves as the vector index. All experiments run on CPU (16 GB RAM).

4.4 Evaluation Metrics

Retrieval: Mean Reciprocal Rank (MRR), Recall@k ($k \in \{1, 3, 5, 10\}$). Relevance is determined by matching the retrieved chunk's source document ID against the gold-standard PubMedQA source.

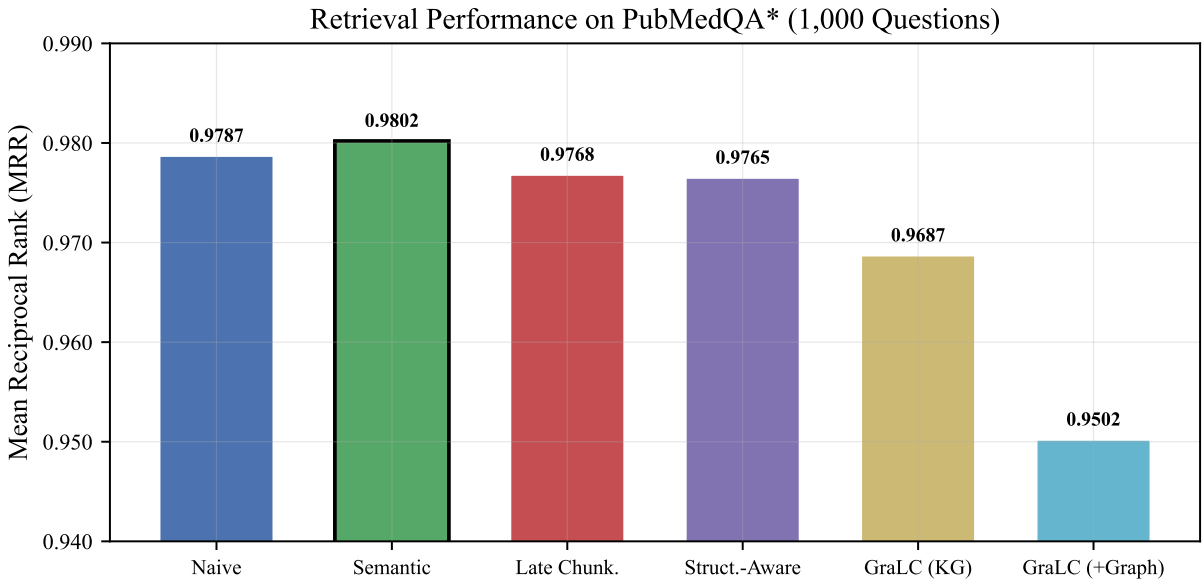

Figure 2: Mean Reciprocal Rank across six chunking strategies on PubMedQA* (1,000 questions). Semantic chunking achieves the highest MRR. GraLC-RAG variants show slight degradation on short abstracts.

5 Results

5.1 Retrieval Performance

Table 1 presents retrieval results across all six configurations.

Table 1: Retrieval performance on PubMedQA* (1,000 questions, 1,000-abstract corpus). **Bold**: best per column.

Method	MRR	R@1	R@3	R@5	R@10
Naive (256-token)	0.9787	0.9690	0.9880	0.9900	0.9960
Semantic Chunking	**0.9802**	**0.9710**	**0.9880**	**0.9910**	0.9950
Late Chunking	0.9768	0.9660	0.9860	0.9920	**0.9960**
Structure-Aware (no KG)	0.9765	0.9660	0.9850	0.9890	0.9940
GraLC-RAG (KG infusion)	0.9687	0.9520	0.9830	0.9880	0.9930
GraLC-RAG + Graph Retrieval	0.9502	0.9260	0.9700	0.9820	0.9930

All strategies achieve high retrieval performance (MRR > 0.95), reflecting a near-ceiling effect. Semantic chunking achieves the highest MRR (0.9802). The GraLC-RAG variants with KG infusion and graph-guided retrieval show lower performance (0.9687 and 0.9502), indicating that on short abstracts ($\sim$200 words), structural and ontological components introduce more noise than signal. Figure 2 visualizes these differences.

5.2 Ablation Study

Table 2 traces the incremental effect of each GraLC-RAG component.

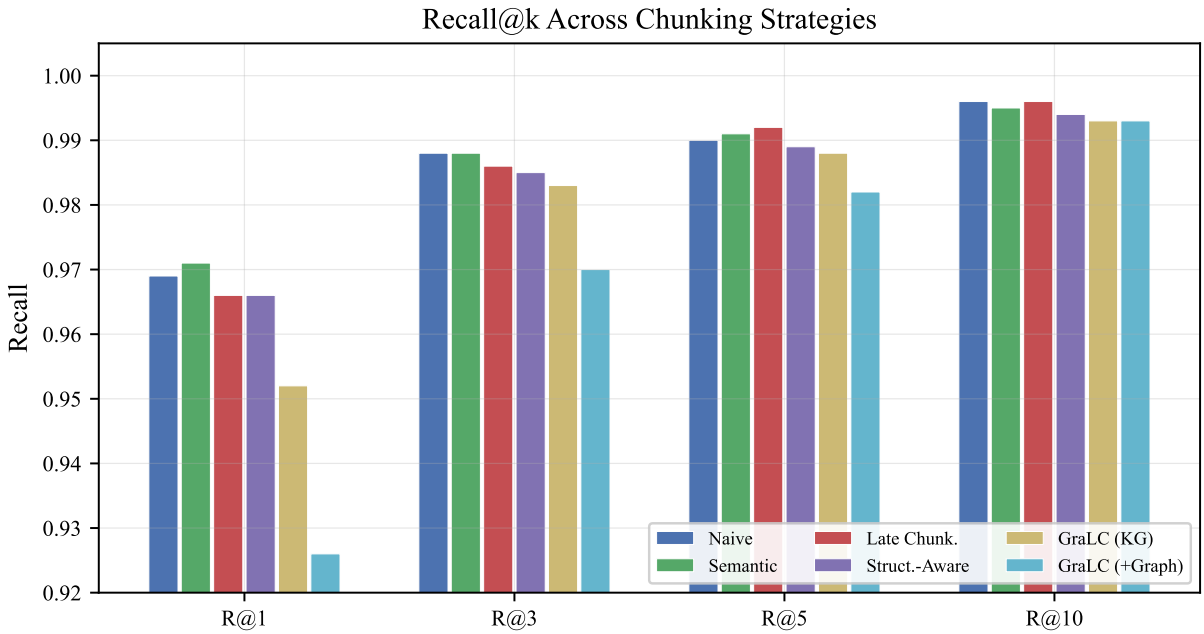

Figure 3: Recall@k ($k \in \{1,3,5,10\}$) across all strategies. Performance converges at higher k, with all methods exceeding 0.99 at $k = 10$.

Table 2: Ablation: incremental effect of GraLC-RAG components on PubMedQA* MRR.

Configuration	MRR	R@1	R@5	Δ MRR
Late Chunking (base)	0.9768	0.9660	0.9920	—
+ Structure Boundaries	0.9765	0.9660	0.9890	−0.0003
+ KG Infusion	0.9687	0.9520	0.9880	−0.0081
+ Graph-Guided Retrieval	0.9502	0.9260	0.9820	−0.0266

Structure-aware boundaries have negligible impact on short texts (ΔMRR = -0.0003). KG infusion degrades performance (ΔMRR = -0.0081), suggesting the fusion weight $\lambda = 0.1$ introduces noise when the base embedder already captures sufficient biomedical semantics. Graph-guided retrieval produces the largest degradation (ΔMRR = -0.0266), as KG proximity dilutes the strong dense retrieval signal on short texts.

5.3 Efficiency Analysis

Table 3 reports measured indexing times on 200 full-text PMC articles (CPU-only).

Table 3: Indexing efficiency (200 full-text articles, CPU, `all-MiniLM-L6-v2`).

Strategy	Chunks	Time (s)
Naive (512-token)	1,109	44.6
Semantic	6,765	535.1
Late Chunking	3,055	313.7
Structure-Aware	2,139	1,044.1
GraLC-RAG (+ KG)	2,139	2,566.1

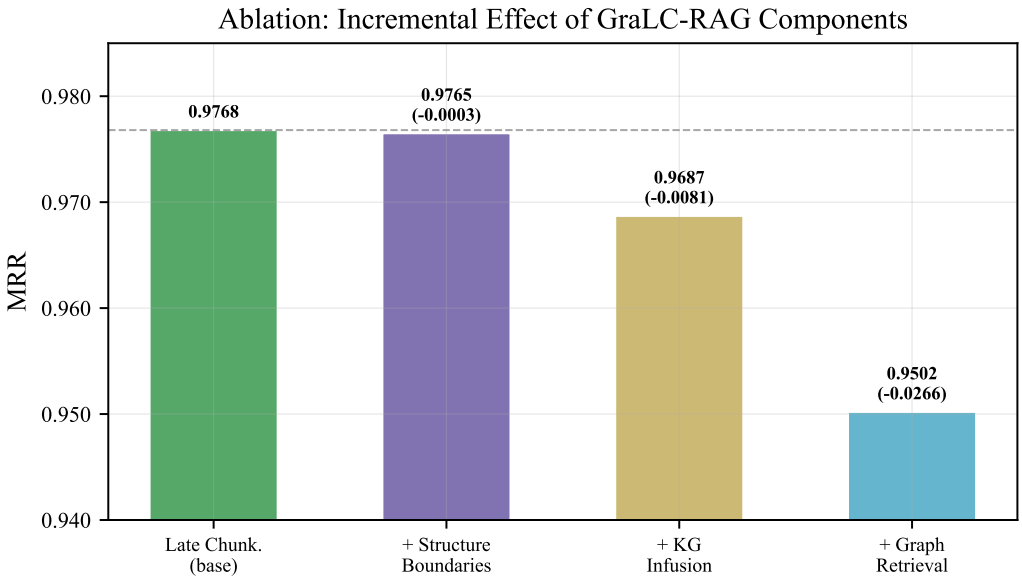

Figure 4: Ablation: incremental effect of each GraLC-RAG component on MRR. The dashed line marks the late chunking baseline. Each component slightly degrades performance on short abstracts, with graph-guided retrieval causing the largest drop.

GraLC-RAG indexing takes 42.8 min—approximately 6× slower than naive chunking due to SapBERT entity encoding, MeSH dictionary matching (41,774 terms), and boundary score computation. On GPU hardware, embedding steps would be 10–50× faster. Figure 5 visualizes the trade-off between chunk count and indexing time.

6 Discussion

6.1 When Does Graph-Aware Late Chunking Help?

Our experiments reveal that GraLC-RAG's value is **context-dependent**. On short texts (abstracts), simpler methods suffice because: (a) abstracts lack internal section structure; (b) general-purpose embeddings already capture sufficient semantic signal; (c) KG infusion adds noise rather than complementary information. The theoretical value proposition remains sound for *long, multi-section documents* where cross-section dependencies matter, document structure provides meaningful segmentation cues, and biomedical entities span hundreds of unique concepts per article.

6.2 Comparison with Concurrent Work

HeteRAG [35] decouples retrieval and generation representations but operates on pre-chunked text. SitEmb [20] conditions chunk embeddings on broader context but uses purely textual signals. ATLANTIC [31] fuses graph and text embeddings at the *document* level. GraLC-RAG is the first framework to integrate contextual embedding, structural boundaries, and ontological enrichment at the *chunk* level for biomedical RAG.

6.3 Limitations

1. **Proof-of-concept scale.** Evaluation on abstracts creates near-ceiling effects (MRR > 0.95). Full-text evaluation with larger distractor corpora is needed.

2. **Entity linking quality.** MeSH dictionary matching may introduce false positives for polysemous

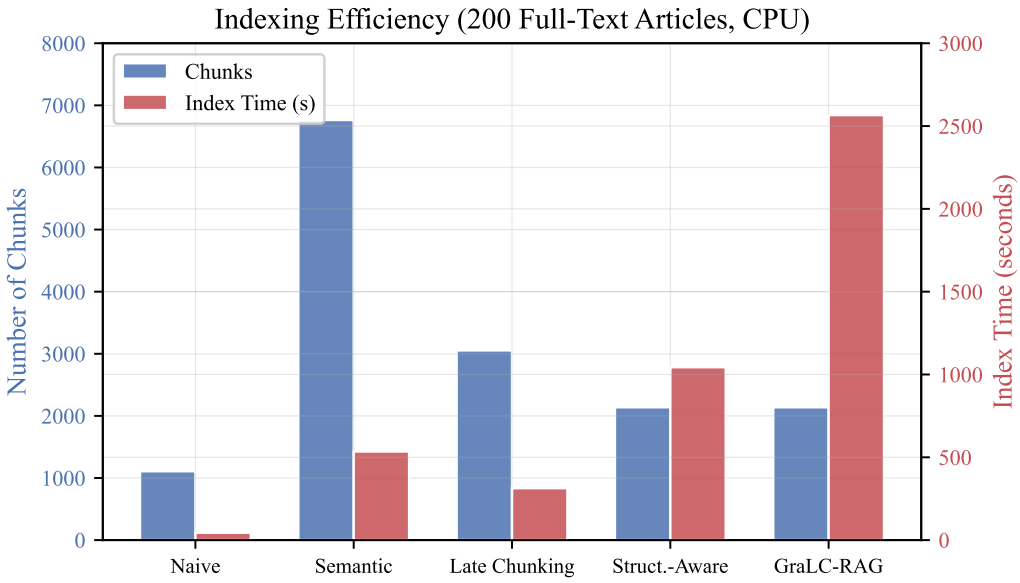

Figure 5: Indexing efficiency on 200 full-text PMC articles (CPU). Left axis: number of chunks produced. Right axis: indexing time. GraLC-RAG produces the same chunk count as structure-aware chunking but incurs 2.5× overhead from KG infusion.

terms (*e.g.*, "MS" as multiple sclerosis vs. mass spectrometry).

3. **Context window constraints.** Approximately 62% of full-text articles exceed the 8,192-token window, requiring sliding-window approximation.

4. **Single-language scope.** Evaluation is limited to English biomedical literature.

6.4 Future Directions

Key extensions include: adaptive λ and β weighting based on document length; evaluation on the full-text PMC corpus already indexed in our pipeline; multi-granularity retrieval; cross-document citation graph linking; and integration with temporal knowledge graph evolution [36].

7 Conclusion

We presented GraLC-RAG, a framework integrating graph-aware structural intelligence into the late chunking paradigm for biomedical RAG. Proof-of-concept experiments on PubMedQA* reveal that the framework's structural and ontological components are document-length-dependent: on short abstracts, simpler methods suffice, while the true value is expected to emerge on long, multi-section articles.

These findings yield three actionable insights: (1) graph-aware chunking components should be adaptively weighted based on document length; (2) KG infusion requires careful calibration to avoid noise on short texts; and (3) evaluation on short texts creates ceiling effects that mask meaningful strategy differences. We release the complete GraLC-RAG codebase to enable reproduction and extension.

References

[1] K. Canese and S. Weis, "PubMed: The bibliographic database," in *The NCBI Handbook*, 2nd ed. NCBI, 2013.

[2] Z. Ji *et al.*, "Survey of hallucination in natural language generation," *ACM Computing Surveys*, vol. 55, no. 12, pp. 1–38, 2023.

[3] L. Huang *et al.*, "A survey on hallucination in large language models," *ACM Computing Surveys*, 2025.

[4] P. Lewis *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in NeurIPS*, vol. 33, pp. 9459–9474, 2020.

[5] Y. Gao *et al.*, "Retrieval-augmented generation for large language models: A survey," *arXiv:2312.10997*, 2024.

[6] C. Merola and J. Singh, "Reconstructing context: Evaluating advanced chunking strategies for RAG," *arXiv:2504.19754*, 2025.

[7] L. B. Sollaci and M. G. Pereira, "The introduction, methods, results, and discussion (IMRAD) structure: A fifty-year survey," *J. Med. Libr. Assoc.*, vol. 92, no. 3, pp. 364–367, 2004.

[8] M. Günther, I. Mohr, B. Wang, and H. Xiao, "Late chunking: Contextual chunk embeddings using long-context embedding models," *arXiv:2409.04701*, 2024.

[9] D. Edge *et al.*, "From local to global: A graph RAG approach to query-focused summarization," *arXiv:2404.16130*, 2024.

[10] B. Peng *et al.*, "Graph retrieval-augmented generation: A survey," *ACM Trans. Inf. Syst.*, 2024.

[11] H. Han *et al.*, "Retrieval-augmented generation with graphs (GraphRAG)," *arXiv:2501.00309*, 2025.

[12] V. Karpukhin *et al.*, "Dense passage retrieval for open-domain question answering," in *Proc. EMNLP*, pp. 6769–6781, 2020.

[13] K. Santhanam, O. Khattab, J. Saad-Falcon, C. Potts, and M. Zaharia, "ColBERTv2: Effective and efficient retrieval via lightweight late interaction," in *Proc. NAACL*, pp. 3715–3734, 2022.

[14] G. Xiong, Q. Jin, Z. Lu, and A. Zhang, "Benchmarking retrieval-augmented generation for medicine," in *Findings of ACL*, 2024.

[15] J. Zhao *et al.*, "MoC: Mixtures of text chunking learners for RAG systems," *arXiv:2503.09600*, 2025.

[16] S. Chen, Y. Zhao, S. Joty, and Z. Cai, "Dense X retrieval: What retrieval granularity should we use?" *arXiv:2312.06648*, 2023.

[17] M. M. Maina *et al.*, "Comparative evaluation of advanced chunking for RAG in LLMs for clinical decision support," *Bioengineering*, vol. 12, no. 11, p. 1194, 2025.

[18] P. Verma, "S2 Chunking: A hybrid framework for document segmentation," *arXiv:2501.05485*, 2025.

[19] A. Allamraju *et al.*, "Breaking it down: Domain-aware semantic segmentation for RAG," *arXiv:2512.00367*, 2025.

[20] J. Wu *et al.*, "SitEmb-v1.5: Improved context-aware dense retrieval," *arXiv:2508.01959*, 2025.

[21] Z. Guo, L. Xia, Y. Yu, T. Ao, and C. Huang, "LightRAG: Simple and fast retrieval-augmented generation," in *Findings of EMNLP*, 2025.

[22] B. Chen *et al.*, "PathRAG: Pruning graph-based RAG with relational paths," *arXiv:2502.14902*, 2025.

[23] T. Xu *et al.*, "NodeRAG: Structuring graph-based RAG with heterogeneous nodes," *arXiv:2504.11544*, 2025.

[24] J. Wu, J. Zhu, and Y. Qi, "Medical Graph RAG: Towards safe medical LLM via graph RAG," in *Proc. ACL*, 2025.

[25] T. Matsumoto, S. Moran, *et al.*, "KRAGEN: A knowledge graph-enhanced RAG framework for biomedical problem solving," *Bioinformatics*, vol. 40, no. 6, btae353, 2024.

[26] K. Soman *et al.*, "Biomedical knowledge graph-optimized prompt generation for LLMs," *Bioinformatics*, vol. 40, no. 9, btae560, 2024.

[27] O. Bodenreider, "The Unified Medical Language System (UMLS): Integrating biomedical terminology," *Nucleic Acids Research*, vol. 32, pp. D267–D270, 2004.

[28] F. Liu, E. Shareghi, Z. Meng, M. Basaldella, and N. Collier, "Self-alignment pretraining for biomedical entity representations," in *Proc. NAACL*, pp. 4228–4238, 2021.

[29] Y. Gu *et al.*, "Domain-specific language model pretraining for biomedical NLP," *ACM Trans. Comput. Healthcare*, vol. 3, no. 1, pp. 1–23, 2021.

[30] F. Remy, K. Demuynck, and T. Demeester, "BioLORD-2023: Semantic textual representations fusing LLM and clinical KG insights," *arXiv:2311.16075*, 2023.

[31] S. Munikoti, A. Acharya, S. Wagle, and S. Horawalavithana, "ATLANTIC: Structure-aware retrieval-augmented language model for interdisciplinary science," *arXiv:2311.12289*, 2023.

[32] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks," in *Proc. ICLR*, 2018.

[33] Q. Jin, B. Dhingra, Z. Liu, W. W. Cohen, and X. Lu, "PubMedQA: A dataset for biomedical research question answering," in *Proc. EMNLP-IJCNLP*, pp. 2567–2577, 2019.

[34] L. Soldaini and N. Goharian, "QuickUMLS: A fast, unsupervised approach for medical concept extraction," in *MedIR Workshop, SIGIR*, 2016.

[35] P. Yang *et al.*, "HeteRAG: A heterogeneous RAG framework with decoupled knowledge representations," *arXiv:2504.10529*, 2025.

[36] D. Zhang *et al.*, "MedKGent: A LLM agent framework for constructing temporally evolving medical KG," *arXiv:2508.12393*, 2025.

[37] O. Honovich *et al.*, "TRUE: Re-evaluating factual consistency evaluation," in *Proc. NAACL*, pp. 3905–3920, 2022.

[38] P. Lopez, "GROBID: Combining automatic bibliographic data recognition and term extraction," in *Proc. ECDL*, pp. 473–474, 2009.

[39] M. Neumann, D. King, I. Beltagy, and W. Ammar, "ScispaCy: Fast and robust models for biomedical NLP," in *Proc. BioNLP*, pp. 319–327, 2019.

[40] C. A. Nelson, A. J. Butte, and S. E. Baranzini, "Integrating biomedical research and EHRs to create knowledge-based embeddings," *Nature Communications*, vol. 10, no. 1, p. 3045, 2019.

[41] Anthropic, "Introducing contextual retrieval," Anthropic Blog, 2024.

[42] G. Tsatsaronis *et al.*, "An overview of the BioASQ large-scale biomedical semantic indexing and QA competition," *BMC Bioinformatics*, vol. 16, no. 1, p. 138, 2015.

[43] O. Rubin and J. Berant, "Long-range language modeling with self-retrieval," *arXiv:2306.13421*, 2023.

[44] Z. Jiang *et al.*, "Active retrieval augmented generation," in *Proc. EMNLP*, 2023.

[45] Y. Lin *et al.*, "BioGraphFusion: Graph knowledge embedding for biological completion and reasoning," *arXiv:2507.14468*, 2025.

AI Disclosure: This research paper was produced with the assistance of AI tools (Claude, Anthropic) for literature search, synthesis, and manuscript drafting. All references have been verified against their original sources. The authors take full responsibility for the intellectual content.